ATS Automation - Technology Stack

Core RPA Toolstack

Tool / Library	Use Case	Why No Credentials Needed
Playwright / Selenium	Multi-tab browser automation, form filling, navigation	Uses agent's existing authenticated browser session
Chrome Extension (Content Scripts)	DOM reading, overlay injection, event detection	Runs inside the browser, reads page content directly
PyAutoGUI / AutoIt	Desktop-level clicks, keystrokes, hotkey triggers	Simulates user input - no system access required
Tesseract OCR	Read data from screens, PDFs, portal images	Purely local image processing, offline capable
Python watchdog	Monitor Downloads folder for CSV/PDF exports	File system watcher, no network access needed
pdfplumber / pdfrw	Read and write PDF form fields	Local file manipulation, no PDF service needed
DistilBERT / spaCy	Note summarization, call classification	Model runs on-device - no API calls, no data sent externally
LibreTranslate (offline)	Non-English call/note translation	Self-hosted local instance, no cloud translation API

Recommended Tool Versions (2026)

Browser Automation

```
playwright>=1.50.0
selenium>=4.20.0
```

Desktop Automation

```
pyautogui>=0.9.54
pygetwindow>=0.0.9
```

PDF Processing

```
pdfplumber>=0.11.0
pypdf>=5.0.0
pypdf2>=3.0.0
```

OCR

```
pytesseract>=0.3.10
Pillow>=11.0.0
```

NLP (Local)

```
transformers>=4.40.0
torch>=2.3.0
spacy>=3.8.0
```

Chrome Extension

- Manifest V3
- Content Scripts API
- Chrome Storage API for config

Alternative Tools Considered

Browser Automation Alternatives

Tool	Pros	Cons	Recommendation
Playwright	Faster, better waiting, multi-browser	Less mature than Selenium	PRIMARY
Selenium	Mature ecosystem, wide support	Slower, older API	Fallback
Puppeteer	Great for Node.js	Python support limited	Not recommended

Decision: Use **Playwright as primary**, Selenium as fallback for legacy sites.

Desktop Automation Alternatives

Tool	Pros	Cons	Recommendation
PyAutoGUI	Cross-platform, simple API	Can be blocked by some apps	PRIMARY
AutoIt	Windows-only, powerful	Not cross-platform	Windows only
pynput	More control than PyAutoGUI	Less documented	Consider

Decision: Use **PyAutoGUI as primary**, pynput for specific cases.

PDF Alternatives

Tool	Pros	Cons	Recommendation
pdfplumber	Best table extraction	Slower for large PDFs	PRIMARY
pypdf	Fast, lightweight	Limited form handling	Form filling
pdfrw	Good for form fields	Python 2 legacy	Legacy support
Adobe Acrobat JS	Official, powerful	Requires license	Last resort

Decision: Use **pdfplumber for reading**, **pypdf for writing**.

NLP Alternatives

Tool	Pros	Cons	Recommendation
DistilBERT (HuggingFace)	Good balance speed/accuracy	Requires download	PRIMARY
spaCy	Fast, production-ready	Less flexible	NER tasks
TensorFlow Lite	Mobile-friendly	Limited model options	Not needed

Decision: Use **DistilBERT for summarization**, **spaCy for NER**.

Client-Specific DOM Profiles Required

Flyland Recovery

```
{
  "client": "flyland",
  "systems": {
    "ctm": {
      "url": "*.calltrackingmetrics.co",
      "selectors": {
        "call_notification": ".caller-id",
        "disposition_dropdown": "#disposition",
        "phone_number": ".phone-field"
      }
    },
    "salesforce": {
      "url": "*.flyland.my.salesforce.com",
      "selectors": {
        "account_name": ".accountName",
        "contact_history": ".historySection"
      }
    },
    "zoho": {
      "url": "*.zoho.com/mail",
      "selectors": {
```

```
        "chat_input": "#chatInput",
        "message_thread": ".msgThread"
      }
    }
  }
}
```

Legacy

```
{
  "client": "legacy",
  "systems": {
    "zoho_salesiq": {
      "url": "*.zoho.com/salesiq",
      "selectors": {
        "chat_input": ".zsiq-input",
        "message_thread": ".zsiq-msgcntnr"
      }
    }
  }
}
```

TBT

```
{
  "client": "tbt",
  "note": "NO CTM - Uses SF Lightning exclusively",
  "systems": {
    "sf_lightning": {
      "url": "*.lightning.force.com",
      "selectors": {
        "lead_queue": ".forceTable",
        "call_button": "[title='Call']"
      }
    }
  }
}
```

Banyan

```
{
  "client": "banyan",
  "note": "Tracker is Google Form - use URL pre-fill",
  "systems": {
    "google_form": {
      "url": "*.google.com/forms/*",
      "prefill_pattern": "?entry.FIELD_ID=value",
      "alternative": "direct_sheet_write"
    }
```

```
  }
}
```

Takahami

```
{
  "client": "takami",
  "note": "Uses RingCentral (not CTM)",
  "systems": {
    "ringcentral": {
      "url": "*.ringcentral.com",
      "selectors": {
        "active_call": ".callActive",
        "caller_id": ".callerInfo"
      }
    },
    "kipu": {
      "url": "*.kipuworks.com",
      "selectors": {
        "patient_record": ".patientHeader"
      }
    },
    "collaboratemd": {
      "url": "*.collaboratemd.com",
      "selectors": {
        "claim_status": ".claimRow"
      }
    }
  }
}
```

Element Medical

```
{
  "client": "element",
  "note": "Use TurboScribe exports as NLP input",
  "systems": {
    "turboscribe": {
      "note": "Read transcript exports from configured folder",
      "watch_folder": "~/Downloads/TurboScribe"
    },
    "availity": {
      "url": "*.availity.com",
      "selectors": {
        "eligibility_result": ".eligCard"
      }
    }
  }
}
```

File Structure

```
ats-automation/
├── requirements.txt
├── pyproject.toml
├── README.md
│
├── src/
│   ├── __init__.py
│   ├── chrome_extension/
│   │   ├── manifest.json
│   │   ├── content_scripts/
│   │   │   ├── __init__.py
│   │   │   ├── ctm_monitor.py
│   │   │   ├── sf_reader.py
│   │   │   ├── zoho_chat.py
│   │   │   └── overlay.py
│   │   ├── popup/
│   │   │   ├── popup.html
│   │   │   └── popup.js
│   │   └── config/
│   │       ├── flyland.json
│   │       ├── legacy.json
│   │       ├── tbt.json
│   │       ├── banyan.json
│   │       ├── takahami.json
│   │       └── element.json
│   │
│   ├── playwright/
│   │   ├── __init__.py
│   │   ├── ctm_sf_auto_access.py
│   │   ├── wrapup_sync.py
│   │   ├── pdf_filler.py
│   │   └── fax_automation.py
│   │
│   ├── pyautogui/
│   │   ├── __init__.py
│   │   ├── pdf_filler.py
│   │   ├── fax_hotkeys.py
│   │   └── clipboard_monitor.py
│   │
│   ├── nlp/
│   │   ├── __init__.py
│   │   ├── summarizer.py
│   │   ├── classifier.py
│   │   └── templates/
│   │       ├── flyland_replies.json
│   │       ├── legacy_replies.json
│   │       └── ...
│   │
│   └── utils/
│       ├── __init__.py
```

```
│       ├── file_watcher.py
│       ├── ocr_helper.py
│       └── config_loader.py
│
├── tests/
│   ├── test_chrome_extension/
│   ├── test_playwright/
│   └── test_pyautogui/
│
└── docs/
    ├── deployment.md
    ├── troubleshooting.md
    └── changelog.md
```

Dependencies (requirements.txt)

```
# Core
requests>=2.32.0
python-dotenv>=1.0.0

# Browser Automation
playwright>=1.50.0
selenium>=4.20.0

# Desktop Automation
pyautogui>=0.9.54
pygetwindow>=0.0.9
pynput>=1.7.6

# PDF Processing
pdfplumber>=0.11.0
pypdf>=5.0.0
PyPDF2>=3.0.0

# OCR
pytesseract>=0.3.10
Pillow>=11.0.0

# NLP
transformers>=4.40.0
torch>=2.3.0
spacy>=3.8.0
numpy>=1.26.0

# Utilities
watchdog>=4.0.0
python-dateutil>=2.8.0
```

Installation Commands

```
# Create virtual environment
python3 -m venv ats-env
source ats-env/bin/activate  # or ats-env\Scripts\activate on Windows

# Install dependencies
pip install -r requirements.txt

# Install Playwright browsers
playwright install chromium
playwright install-deps

# Download NLP models
python -c "from transformers import pipeline; pipeline('summarization')"

# Install Chrome Extension
# 1. Open chrome://extensions/
# 2. Enable Developer mode
# 3. Load unpacked -> select chrome_extension folder
```

Last updated: March 2026